



UNIVERSIDAD NACIONAL AUTÓNOMA DE
MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

COMPUTACIÓN GRÁFICA E INTERACCIÓN
HUMANO COMPUTADORA



Ejercicio 05: Optimización y Carga de Modelos

Nombre de la alumna: Macias Niño Carmen Violeta

Nº de Cuenta: 318285450

GRUPO DE LABORATORIO: 02

GRUPO DE TEORÍA: 02

SEMESTRE 2026-2

FECHA DE ENTREGA LÍMITE: 24/03/2026

CALIFICACIÓN: _____

Para la implementación del modelo se utilizaron distintos archivos `.obj`, cada uno correspondiente a una parte del personaje, como cabeza, cuerpo, patas, mandíbula y cola. Estos modelos fueron cargados mediante la clase `Model`, la cual se encarga de leer la información geométrica y almacenarla en buffers para su posterior renderizado.

El uso de múltiples modelos permitió trabajar de forma modular, facilitando la manipulación independiente de cada componente.

Construcción de la escena y uso de transformaciones

Para posicionar los objetos dentro de la escena se utilizó una matriz de transformación denominada `model`, la cual se reinicia constantemente para evitar acumulaciones de transformaciones:

```
model = glm::mat4(1.0f);
```

A partir de esta matriz se aplican transformaciones de traslación y rotación que permiten ubicar y orientar cada objeto en el espacio tridimensional.

Modelo base como referencia

Dentro del código, el modelo completo de Goddard se utiliza como referencia visual para verificar la correcta colocación de las piezas:

```
// ----- INICIA DIBUJO DE LOS DEMAS OBJETOS -----  
// Goddard  
color = glm::vec3(1.0f, 0.0f, 1.0f);  
model = glm::mat4(1.0f);  
model = glm::translate(model, glm::vec3(1.0f, -2.0f, -5.5f));  
model = glm::rotate(model, glm::radians(-30.0f), glm::vec3(0, 1, 0));  
  
glUniform3fv(uniformColor, 1, glm::value_ptr(color));  
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));  
Goddard_M.RenderModel();
```

Este modelo no forma parte de la jerarquía, sino que sirve como guía para ubicar correctamente las demás partes.

Implementación del modelo jerárquico

El modelado jerárquico se implementó tomando como base el cuerpo del personaje, a partir del cual se derivan las demás partes.

Para ello, se utiliza una matriz auxiliar `modelaux`, la cual almacena la transformación del cuerpo:

```

color = glm::vec3(0.0f, 0.0f, 1.0f);
model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(0.5f, -2.0f, -5.5f));
model = glm::translate(model, glm::vec3(0.0f, 0.61f, -1.5f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion15()),
glm::vec3(0.0f, 0.0f, 1.0f));

modelaux = model;

glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Cuerpo_M.RenderModel();

```

Esta matriz permite que todas las partes dependientes hereden la posición y orientación del cuerpo.

Implementación de la cabeza

La cabeza se posiciona a partir de la transformación del cuerpo, asegurando que se mueva junto con él:

```

model = modelaux;
model = glm::translate(model, glm::vec3(1.5f, 0.5f, 0.0f));

glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Cabeza_M.RenderModel();

```

Esto permite que la cabeza mantenga una relación directa con el cuerpo dentro de la jerarquía.

Movimiento de la mandíbula

La mandíbula incluye una transformación adicional de rotación que permite simular la apertura y cierre:

```

model = modelaux;
color = glm::vec3(1.0f, 0.0f, 1.0f);

model = glm::translate(model, glm::vec3(3.0f, 0.75f, 0.065f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion5()),
glm::vec3(0.0f, 0.0f, 1.0f));

glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Mandibula_M.RenderModel();

```

El uso de `getarticulacion5()` permite controlar dinámicamente el movimiento.

Implementación de las patas

Cada pata se implementa a partir de la transformación del cuerpo, aplicando una traslación específica y una rotación controlada por el usuario:

```
// Pata frontal derecha
model = modelaux;
model = glm::translate(model, glm::vec3(1.25f, -0.45f, 0.75f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion1()),
glm::vec3(0.0f, 0.0f, 1.0f));

glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
PataFD_M.RenderModel();

// Pata frontal izquierda
model = modelaux;
model = glm::translate(model, glm::vec3(1.25f, -0.38f, -0.75f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion3()),
glm::vec3(0.0f, 0.0f, 1.0f));

glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
PataFI_M.RenderModel();
```

Cada pata tiene su propia articulación, lo que permite simular movimiento independiente.

Implementación de la cola

La cola también se implementa como una extensión del cuerpo:

```
model = modelaux;
model = glm::translate(model, glm::vec3(-1.67f, -0.1f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion11()),
glm::vec3(0.0f, 1.0f, 0.0f));

glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Cola_M.RenderModel();
```

En este caso, la rotación se realiza sobre el eje Y.

Control de articulaciones

El movimiento de cada parte del modelo se controla mediante funciones del objeto mainWindow, las cuales devuelven valores que representan ángulos de rotación.

Esto permite manipular el modelo mediante el teclado, generando animaciones simples.

Uso de la cámara

La cámara se configura para observar la escena desde una posición adecuada:

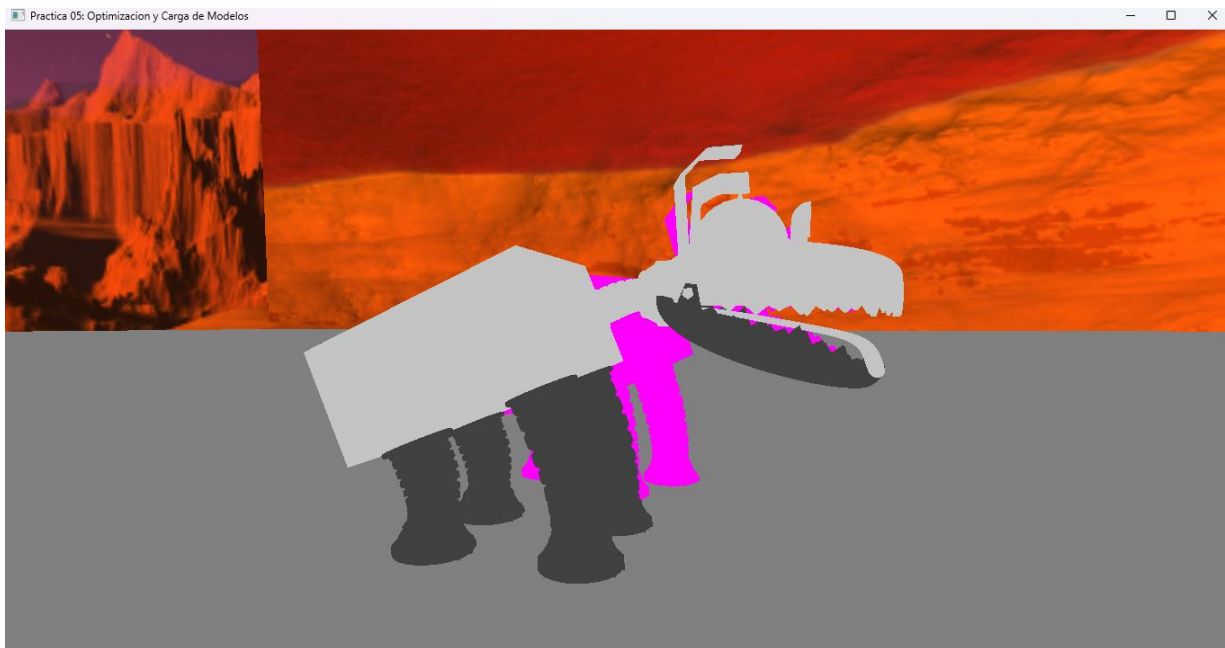
```
camera = Camera(  
    glm::vec3(0.0f, 1.0f, 6.0f),  
    glm::vec3(0.0f, 1.0f, 0.0f),  
    -60.0f,  
    0.0f,  
    0.3f,  
    1.0f  
);
```

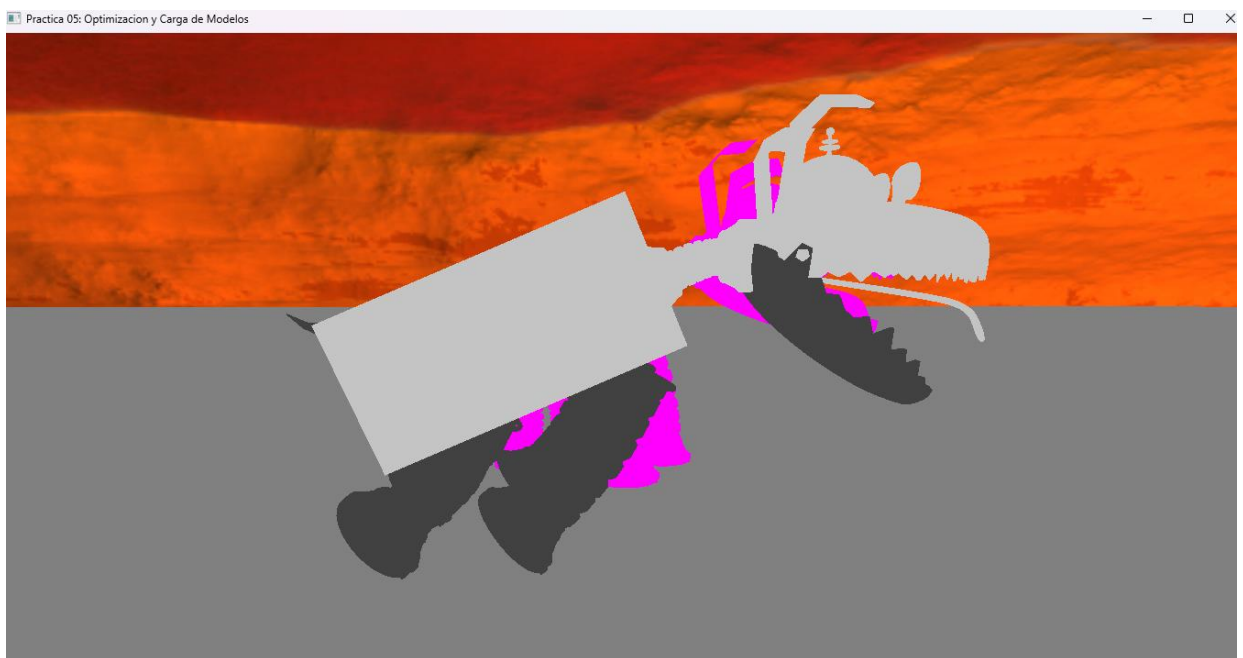
Esto permite tener una vista clara del modelo completo.

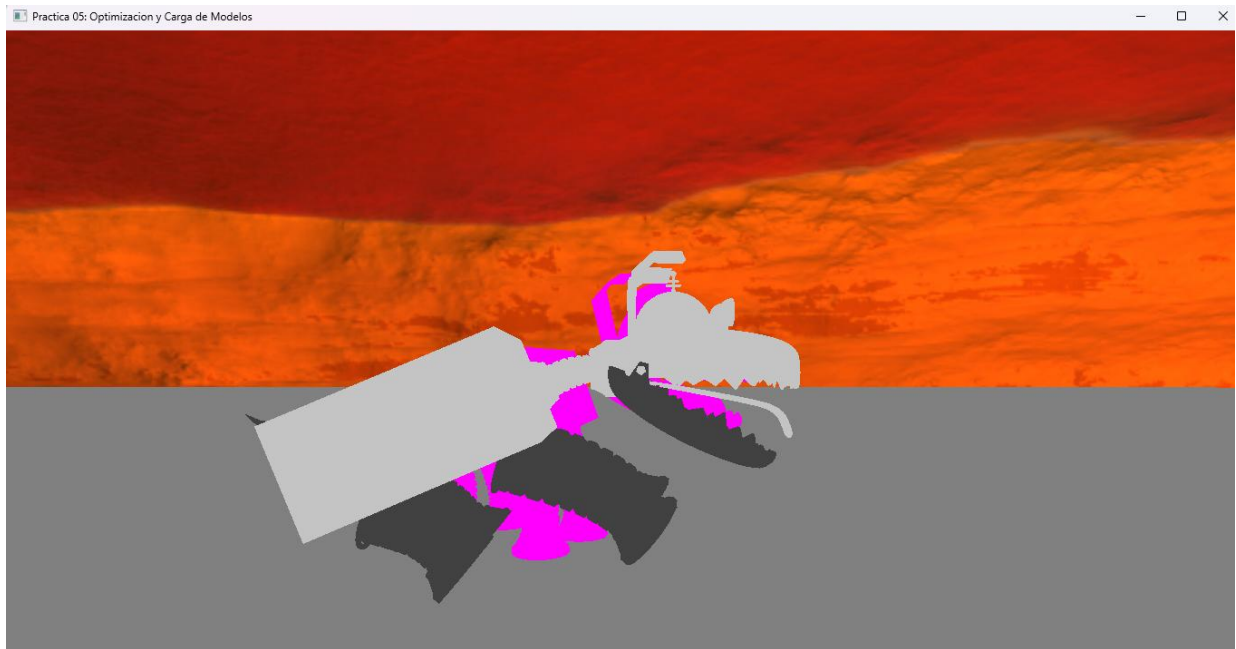
Resultados

Se logró integrar correctamente un modelo 3D compuesto por múltiples partes, aplicando modelado jerárquico y permitiendo el control independiente de sus articulaciones.

El modelo responde a las entradas del usuario y mantiene coherencia en sus movimientos.







Conclusión

Esta práctica me permitió comprender la importancia del modelado jerárquico en gráficos por computadora, así como el uso de transformaciones geométricas para la construcción de modelos complejos.

El uso de matrices auxiliares facilitó la relación entre las distintas partes del modelo, permitiendo generar animaciones estructuradas.